Programa:	ExpressZIP 4.0
Protección:	Time-Trial 15 días
Objetivos:	Evitar que expire
URL descarga:	http://www.avantrix.com
Dificultad:	Bastante sencilla
Herramientas:	OllyDbg v1.10 (que sólo pare en BPM_ejecución)+ PEiD 0.94
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 13-11-05

Hola a todo CracksLatinoS..!!

Qué tal compañeros? Hoy vamos a ver cómo desproteger una aplicación compilada en VisualBasic que viene con una limitación Trial de 15 días y una nagscreen de regalo xDD se trata del "ExpressZIP 4.0" que podéis descargar siguiendo el link arriba indicado, si hubiera alguien que no disponga de conexión o cualquier otro motivo me lo puede solicitar y se lo enviaré al privado sin problemas.

Antes de atacar lo instalaremos y le echaremos un vistazo con PEiD que siempre es bueno:

Como os decía se trata de un VisualBasic, estos programas nos pueden dar dolor de cabeza (en el buen sentido de la palabra) si no los atacamos con las herramientas necesarias, por ejemplo para obtener un serial válido es bueno atacarlo con SmartCheck, y si queremos parchearlo es bueno hacerlo con Olly pero en la versión parcheada por Ricardo el cual sólo para en los BPM por ejecución, esto es buenísimo ya que en Visual los programas se pasan la mayor parte del tiempo en la librería MSVBVMxx.DLL, siendo las xx 50 ó 60 según sea la versión 5 ó 6 respectivamente, entonces lo que haremos será abrir el Olly parcheado y cargar la aplicación en él, como hay varios Ollys parcheados se les puso un nombre a cada uno de ellos, en este caso trabajaremos con el Parcheado5, estamos en el EntryPoint del programa y ya vemos el PUSH inicial y el CALL a la DLL de Visual, esto siempre es así en programas compilados en VB, veamos:

```
00404B30  $  68 B8DA4500  PUSH ExZip.0045DAB8                    ASCII "VB5!6&*"
00404B35  .  E8 F0FFFFFF  CALL <JMP.&MSVBVM60.#100>
00404B3A  .  0000         ADD BYTE PTR DS:[EAX],AL
00404B3C  .  0000         ADD BYTE PTR DS:[EAX],AL
```

Ese CALL nos lleva a este JMP:

```
00404B2A  $- FF25 3012400( JMP DWORD PTR DS:[<&MSVBVM60.#100>]   MSVBVM60.ThunRTMain
00404B30  $  68 B8DA4500   PUSH ExZip.0045DAB8                   ASCII "VB5!6&*"
00404B35  .  E8 F0FFFFFF   CALL <JMP.&MSVBVM60.#100>
00404B3A  .  0000          ADD BYTE PTR DS:[EAX],AL
00404B3C  .  0000          ADD BYTE PTR DS:[EAX],AL
```

Esa llamada a "ThunRTMain" será siempre la misma en programas hechos en Visual y es la función principal, a partir de ella se llamarán al resto de funciones que el programa importa, si pusiéramos un BPM en la primera sección en un Olly normal (sin parchear) nos pasaríamos horas esperando encontrar la zona clave ya que pararía muchísimas veces por lectura y/o escritura, en este caso no será así, jeje.. ya veréis, pongamos el BPM:

```
003D0000 00003000                          Priv RW  RW
00400000 00001000 ExZip          PE header Imag R   RWE
00401000 001B1000 ExZip  .text   cod
005B2000 00013000 ExZip  .data   dat
005C5000 00001000 ExZip  .rsrc   res
005D0000 00006000
00690000 00002000
006A0000 00103000
007B0000 000CD000
00AB0000 00010000
00EB0000 00003000
62E30000 00001000 LPK            PE
62E31000 00004000 LPK    .text   cod
62E35000 00001000 LPK    .data   dat
62E36000 00001000 LPK    .rsrc   res
62E37000 00001000 LPK    .reloc  rel
66000000 00001000 MSVBVM60       PE
66001000 000FC000 MSVBVM60 .text cod
660FD000 0000D000 MSVBVM60 ENGINE cod
```

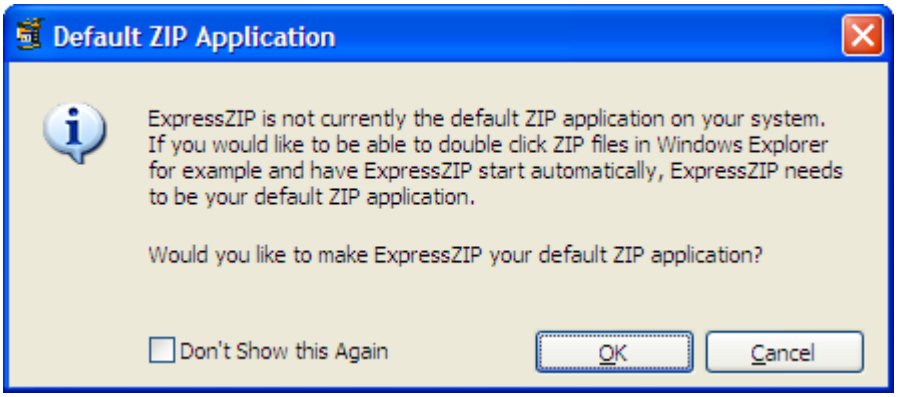
Actualize
View in Disassembler — Enter
Dump in CPU
Dump
Search — Ctrl+B
Set break-on-access — F2
Set memory breakpoint on access
Set memory breakpoint on write

Esa es la primera sección sin contar la cabecera (PE header) dejamos el BPM puesto y damos RUN (pulsando F9) si doy detalles así es porque como sabemos se ha incorporado gente nueva a la lista y no se le puede dar nada por sabido sino que hay que ayudarles, los que saben más de esto deberán entenderlo ;-) y el BPM irá haciendo su trabajo, irá parando sólo por ejecución que es lo que nos interesa, cada vez que pare iremos traceando con F8 para no entrar en los CALL de momento y cuando retorne a la librería de Visual volvemos a pulsar F9 y así caemos de nuevo en el código del ejecutable, esta

jugada se repetirá tantas veces como haga falta hasta ir descubriendo los puntos calientes, a veces se tarda poco y otras se tarda más, todo depende, en este caso una de las veces que para es aquí:

```
00571340  .  894A 0C      MOV DWORD PTR DS:[EDX+C],ECX
00571343  .  8B95 68FFFFF MOV EDX,DWORD PTR SS:[EBP-98]
00571349  .  8B02         MOV EAX,DWORD PTR DS:[EDX]
0057134B  .  8B8D 68FFFFF MOV ECX,DWORD PTR SS:[EBP-98]
00571351  .  51           PUSH ECX
00571352  .  FF90 B002000 CALL DWORD PTR DS:[EAX+2B0]        MSVBVM60.66104348
00571358  .  DBE2         FCLEX
```

bueno para más arriba lo que al pasar el CALL con F8 queda el Olly detenido como si estuviera colgado, pero no es así lo que ocurre es que nos ha aparecido este cuadro en pantalla y está a la espera de que hagamos algo:



Si tenemos ventanas abiertas deberemos minimizarlas para poder verlo ya que se queda en segundo plano, lo que nos pregunta es si queremos dejar la aplicación como predeterminada para el tratamiento de archivos ZIP, yo elijo que no y tildo la opción "Don't show this Again" para que no lo vuelva a mostrar más, al cerrarlo nos devuelve el control en Olly en la siguiente instrucción, entonces seguimos traceando con F8, salta de nuevo a la librería de visual pero nada, F9 y de nuevo en el código del ejecutable, jeje.. como mola esto, vemos además como el baner inicial ha salido ya que lo hemos visto aparecer ligeramente en pantalla (como un parpadeo), poco después llegamos a un CALL donde parece ejecutar bastante código:

```
005716EA  .  8941 04      MOV DWORD PTR DS:[ECX+4],EAX
005716ED  .  8951 08      MOV DWORD PTR DS:[ECX+8],EDX
005716F0  .  8B55 E4      MOV EDX,DWORD PTR SS:[EBP-1C]
005716F3  .  8951 0C      MOV DWORD PTR DS:[ECX+C],EDX
005716F6  .  FF97 B002000 CALL DWORD PTR DS:[EDI+2B0]        ;ojo a este CALL...
005716FC  .  85C0         TEST EAX,EAX
005716FE  .  DBE2         FCLEX
00571700  .v 7D 16        JGE SHORT ExZip.00571718
00571702  .  8B1D 7C10400 MOV EBX,DWORD PTR DS:[<&MSVBVM60.__vbaH  MSVBVM60.__vbaHresultChecl
```

Incluso nos mete en una excepción, quedamos detenidos aquí cuando lo superamos con F8:

```
77F410A2  v  E9 BD000000  JMP ntdll.77F41164
77F410A7     55           PUSH EBP
77F410A8     8BEC         MOV EBP,ESP
77F410AA     9C           PUSHFD
77F410AB     81EC D0020000 SUB ESP,2D0
77F410B1     8985 DCFDFFFF MOV DWORD PTR SS:[EBP-224],EAX
```

Estamos en ntdll.dll y la llamada es a RaiseException, más abajo se puede ver aunque en la imagen no la veamos está allí, si pulsamos F9 caemos aquí:

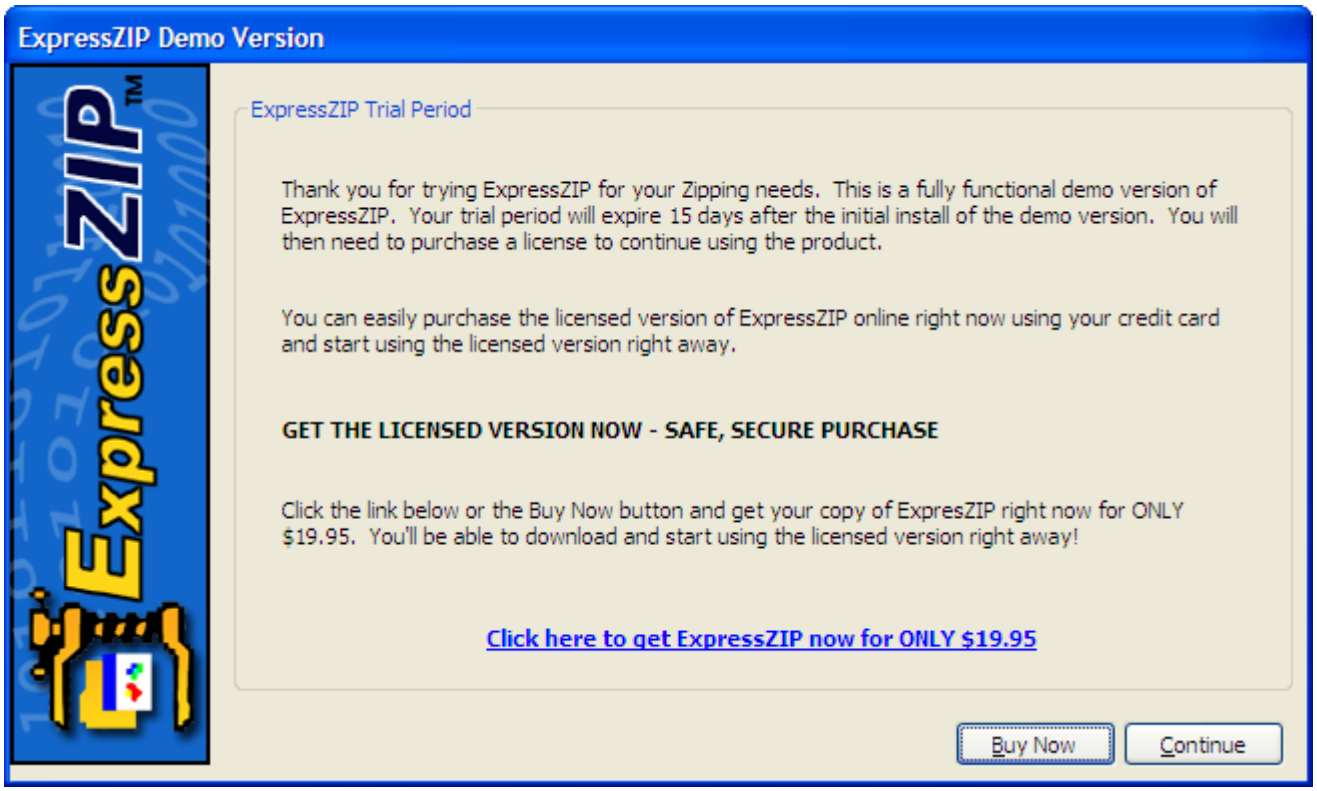
```
0054753C  .  51           PUSH ECX
0054753D  .  6A 04        PUSH 4
0054753F  .  FF15 3410400 CALL DWORD PTR DS:[<&MSVBVM60.__vbaFree  MSVBVM60.__vbaFreeVarList
00547545  .  83C4 14      ADD ESP,14
00547548  .  FF15 3810400 CALL DWORD PTR DS:[<&MSVBVM60.__vbaEnd>  MSVBVM60.__vbaEnd
0054754E  >  E8 0903F3FF  CALL ExZip.0047785C
00547553  .  8BF0         MOV ESI,EAX
00547555  .  FF15 7410400 CALL DWORD PTR DS:[<&MSVBVM60.__vbaSetS  MSVBVM60.__vbaSetSystemEr
0054755B  .  83FE 01      CMP ESI,1
0054755E  .v 0F84 1801000 JE ExZip.0054767C
00547564  .  391D 7C315B0 CMP DWORD PTR DS:[5B317C],EBX
0054756A  .v 75 10        JNZ SHORT ExZip.0054757C
0054756C  .  68 7C315B00  PUSH ExZip.005B317C
```

A esta zona llegamos después que se produzca la excepción vete a saber porqué motivo, hay veces en que el programador la coloca a propósito para saber si la aplicación está siendo depurada bajo algún debugger tipo Olly, SoftIce, TRW, syser, etc., si lo está se dará cuenta e iremos a parar a una parte de código no desencriptada o bien a una zona donde nos mete en una excepción no continuable, etc., en fin las artimañas que se les ocurran a los programadores de la misma, todos sabemos que no les gusta que trasteemos sus creaciones, jeje…

Tras superar esta zona y caer de nuevo en la dll de visual pulsamos F9 y llegamos al código del .exe nuevamente, seguimos con F8 y nos paramos en este CALL:

```
0054A8EB  .  8B8D 70FFFFF[ MOV ECX,DWORD PTR SS:[EBP-90]
0054A8F1  .  8B11          MOV EDX,DWORD PTR DS:[ECX]
0054A8F3  .  8B85 70FFFFF[ MOV EAX,DWORD PTR SS:[EBP-90]
0054A8F9  .  50            PUSH EAX
0054A8FA  .  FF92 B002000( CALL DWORD PTR DS:[EDX+2B0]      ;Aqui paramos...
0054A900  .  DBE2          FCLEX
0054A902  .  8985 6CFFFFF[ MOV DWORD PTR SS:[EBP-94],EAX
```

Si vamos al escritorio vemos que nos ha aparecido la nag de principio:



Anda que es pequeña eh! Es tamaño familiar como las pizzas jeje.. nada aceptamos y vovemos a Olly, si miramos los registros vemos:

```
Registers (FPU)
EAX 00000000
ECX 77D19E48 USER32.77D19E48
EDX 00000AC2
EBX 00000001
ESP 0012F5E4
EBP 0012F8A4
ESI 0012F980
EDI 0012F8B0
```

EAX==0 =>Valor de retorno de la función..!!

O sea que al pulsar Continue nos retorna EAX==0 esto es interesante saberlo, para poder nopear el call y evitar la nag debemos tener cuidado con la pila (registro ESP) tomamos los valores de este registro antes y desués de pasar por dicho call y obtenemos:

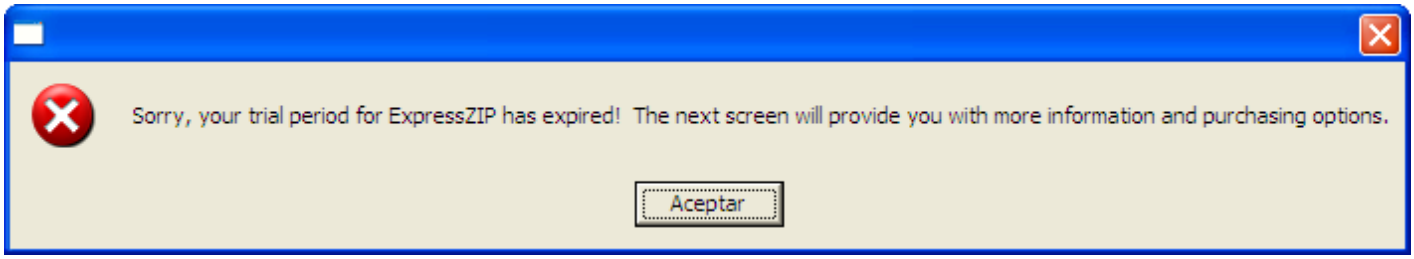
antes del call ESP==12F5C0
después del call ESP==12F5E4
desajuste de +24h

Es decir que deberemos sumarle 24 unidades para no dejarla descuadrada ya que el programa no funcionará, entonces o bien le sumamos estas 24 uds. O bien nopeamos los PUSH que acompañan a dicho CALL, un posible cambio puede ser este:

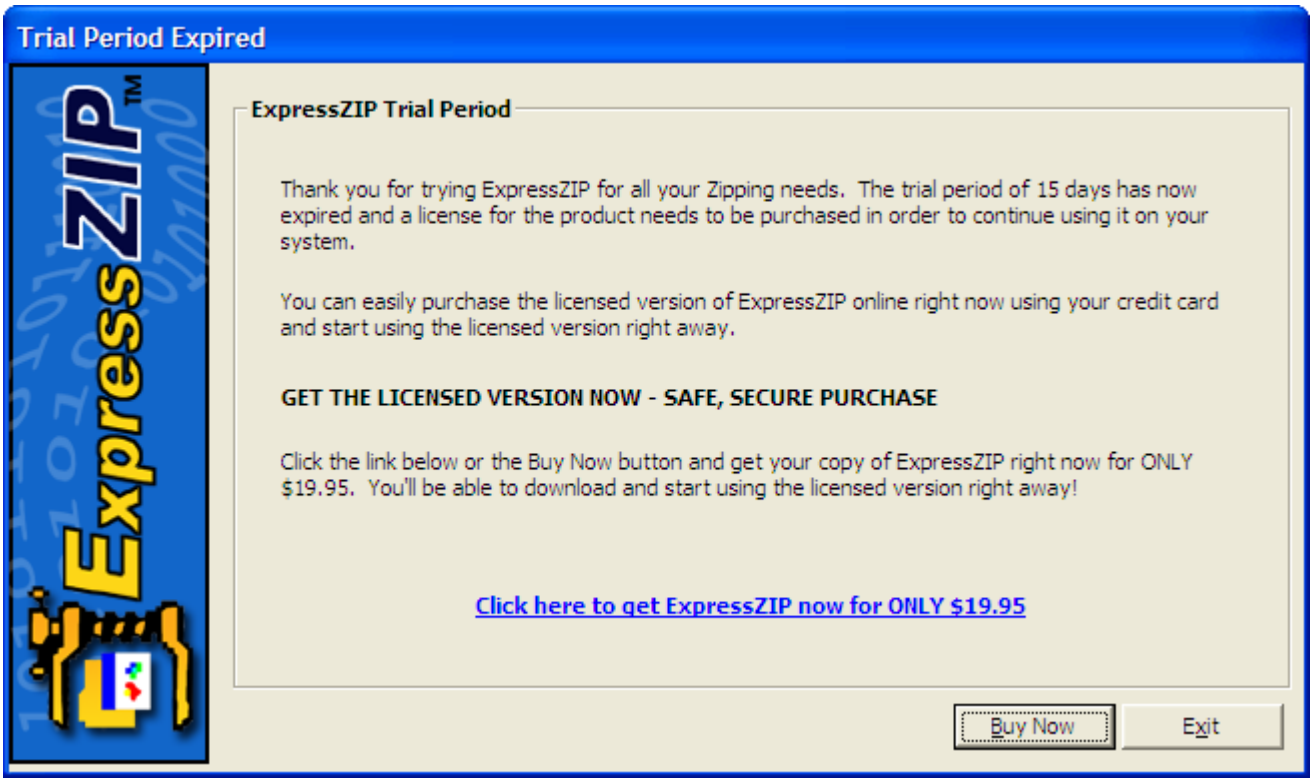
```
0054A8F9  .  50            PUSH EAX
0054A8FA     83C4 24       ADD ESP,24          ;reajustamos la pila
0054A8FD     33C0          XOR EAX,EAX         ;dejamos EAX==0
0054A8FF     90            NOP
0054A900  .  DBE2          FCLEX
0054A902  .  8985 6CFFFFFF MOV DWORD PTR SS:[EBP-94],EAX
```

Sustituimos el CALL por un “ADD ESP,24” que reajusta la pila por nosotros simulando que se ha entrado en el CALL y después ponemos EAX a cero como hacía el propio call, de esta manera no podrá saber que éste ni si quiera se ejecutó, salvamos los cambios al disco como parche.exe, lo probamos y parece funcionar bien ;-)

Pero… adelantemos 1 año el reloj del sistema y ejecutemos el crack:



Vaya parece que nos pilló por alguna parte, si aceptamos el mensaje nos saca una enorme nag como la anterior pero que nos dice otras cosas:



Lo que haremos será exactamente lo mismo que hicimos antes, es decir poner el BPm en la primera sección e ir esperando a que pare en ella, tracearla en busca del CALLresponsable del mensaje y de la nag, esto ocurre aquí:

```
005716EA  .  8941 04       MOV DWORD PTR DS:[ECX+4],EAX
005716ED  .  8951 08       MOV DWORD PTR DS:[ECX+8],EDX
005716F0  .  8B55 E4       MOV EDX,DWORD PTR SS:[EBP-1C]
005716F3  .  8951 0C       MOV DWORD PTR DS:[ECX+C],EDX
005716F6  .  FF97 B002000  CALL DWORD PTR DS:[EDI+2B0]        ;ojo a este CALL...
005716FC  .  85C0          TEST EAX,EAX
005716FE  .  DBE2          FCLEX
00571700  .v 7D 16         JGE SHORT ExZip.00571718
00571702  .  8B1D 7C10400  MOV EBX,DWORD PTR DS:[<&MSVBVM60.__vbaH  MSVBVM60.__vbaHresultChecl
```

Recordáis? Es el mismo CALL que nos metía en la excepción el que ahora saca el primer mensaje_expired, después la nag aparece aquí:

```
005475CD  .  8B55 98       MOV EDX,DWORD PTR SS:[EBP-68]
005475D0  .  8951 0C       MOV DWORD PTR DS:[ECX+C],EDX
005475D3  .  56            PUSH ESI
005475D4  .  FF97 B0020000 CALL DWORD PTR DS:[EDI+2B0]        MSVBVM60.66104348
005475DA  .  DBE2          FCLEX
005475DC  .  85C0          TEST EAX,EAX
005475DE  .v 0F8D 92000000 JGE ExZip.00547676
005475E4  .  68 B0020000   PUSH 2B0
```

En este CALL saca la nagscreen y está un poco antes del call que saca la otra nagscreen cuando estamos en periodo trial (lo hacía en :54A8FA, entonces entre ambas direcciones debe existir un salto que decida cual de ellas mostrar, entonces nos desplazamos hacia arriba con la rueda del ratón y miramos los posibles saltos que nos alejen de la dirección :5475D4, encontramos solamente uno antes del inicio de la rutina y es este:

```
00547548  .   FF15 38104000  CALL DWORD PTR DS:[<&MSVBVM60.__vbaEnd>]  MSVBVM60.__vbaEnd
0054754E  >   E8 0903F3FF    CALL ExZip.0047785C
00547553  .   8BF0           MOV ESI,EAX
00547555  .   FF15 74104000  CALL DWORD PTR DS:[<&MSVBVM60.__vbaSetS  MSVBVM60.__vbaSetSystemError
0054755B  .   83FE 01        CMP ESI,1
0054755E  .v  0F84 18010000  JE ExZip.0054767C                        ;evitamos el Trial ;-)
00547564  .   391D 7C315B00  CMP DWORD PTR DS:[5B317C],EBX
0054756A  .v  75 10          JNZ SHORT ExZip.0054757C
0054756C  .   68 7C315B00    PUSH ExZip.005B317C
```

Este JE decide si el programa ha expirado o no, de momento lo jumpeamos:

```
00547555  .   FF15 74104000  CALL DWORD PTR DS:[<&MSVBVM60.__vbaSetS  MSVBVM60.__vbaSetSystemError
0054755B  .   83FE 01        CMP ESI,1
0054755E  v   E9 19010000    JMP ExZip.0054767C                       ;evitamos el Trial ;-)
00547563      90             NOP
00547564  .   391D 7C315B00  CMP DWORD PTR DS:[5B317C],EBX
0054756A  .v  75 10          JNZ SHORT ExZip.0054757C
0054756C  .   68 7C315B00    PUSH ExZip.005B317C
```

Y salvamos los cambios al disco como tute1.exe que ya sabemos que no expira pero nos sigue mostrando el primer mensaje, sabemos además que lo hace en :5716F6, ponemos un BP con F2 en esta dirección y llegamos con F9, allí ponemos un BPM en la primera sección como hicimos antes y vovemos a pulsar F9, cuando para traceamos con F8 en busca del CALL que saca el molesto mensajito:

```
0054A82B  .   8B11           MOV EDX,DWORD PTR DS:[ECX]
0054A82D  .   8B45 08        MOV EAX,DWORD PTR SS:[EBP+8]
0054A830  .   50             PUSH EAX
0054A831  .   FF92 FC060000  CALL DWORD PTR DS:[EDX+6FC]              ;nos saca el mensaje...
0054A837  .   C745 FC 0800000 MOV DWORD PTR SS:[EBP-4],8
0054A83E  .   833D E0315B00 0 CMP DWORD PTR DS:[5B31E0],0
0054A845  .v  75 1C          JNZ SHORT ExZip.0054A863
```

En este CALL lo saca, hacemos como antes en el otro CALL, tomamos el valor de ESP antes y después del call, obtenemos estos resultados:

antes del call ESP==12F5E0
después del call ESP==12F5E4
desajuste de +4h

Es decir que debemos sumarle 4 unidades para no descuadrar la pila, además de esto dentro de este CALL se realizan las comprobaciones del Time-Trial y atacar por aquí nos hubiera simplificado mucho el trabajo ya que nopeando este CALL evitamos pasar por el que hemos modificado anteriormente, ahora da igual, un posible cambio podría ser este:

```
0054A830  .   50             PUSH EAX
0054A831      83C4 04        ADD ESP,4                               ;nos saca el mensaje...
0054A834      90             NOP
0054A835      90             NOP
0054A836      90             NOP
0054A837  .   C745 FC 0800000 MOV DWORD PTR SS:[EBP-4],8
```

Evitamos el CALL y cuadramos ESP añadiéndole 4, pero ahora que hemos evitado que expire volvemos a ver la nag en modo trial, jeje.. esto si que es un bucle verdad? pues nada vuelta a Olly. Trabajamos con el exe modificado con los cambios descritos hasta ahora, lo lanzamos con F9 y el BPM en la primera sección, pero no va a hacer falta, resulta que entre cambio y cambio se nos quedó este CALL si nmodificar:

```
0054A8F9   .   50              PUSH EAX
0054A8FA       FF92 B0020000   CALL DWORD PTR DS:[EDX+2B0]
```

que es precisamente el que nos muestra la nag, lo modificamos como antes:

```
0054A8FA       83C4 24         ADD ESP,24
0054A8FD       33C0            XOR EAX,EAX
0054A8FF       90              NOP
```

Lo de antes reajustamos la pila, dejamos EAX==0 y para adelante, jeje.. salvamos los cambios y programa crackeado ;-)

Para cualquier duda, sugerencia o crítica por su puesto las acepto, estaré en:

syxe00@yahoo.es

Un fuerte abrazo y un saludo para todo CraksLatinoS sin excepción..!! hasta la próxima…